Design and FPGA Implementation of a Scalable SPI Bus Interface Supporting Multiple Sensors with Variable Data Widths

Bhargav Vemparala
Department of Electrical and Computer Engineering
University Evaluation Board
Email: evaluation@university.edu

***Abstract*—The Serial Peripheral Interface (SPI) is a ubiquitous protocol for short-distance communication in embedded systems. However, traditional SPI configurations scale poorly, requiring a dedicated Chip Select (CS) line for each additional peripheral. This $O(N)$ wiring complexity quickly consumes general-purpose input/output (GPIO) resources on the host microcontroller (MCU). This paper presents the design, evolution, and FPGA implementation of a Scalable SPI Bus Interface that acts as an intelligent bridge between a single SPI master and multiple heterogeneous sensors. The proposed interface reduces wiring complexity to a constant 3-wire or 4-wire physical bus by introducing address decoding and dynamic packet handling. We implement and compare two distinct communication methodologies: (1) the Address + Bit-Length Method, where data transfer width is dynamically specified per transaction, and (2) the ROM Lookup Table Method, which utilizes an on-chip ROM containing hardware configurations (widths, start offsets) for each sensor, thereby minimizing transaction overhead. The system is written in synthesizable Verilog HDL, verified via simulation, and implemented on a Xilinx Artix-7 FPGA. To facilitate real-time monitoring and control, an ESP32 microcontroller is integrated to host a web-based user interface, bridging the FPGA-based SPI bus with network-attached users. Results demonstrate a 33.3% reduction in communication latency for the ROM-based method compared to the dynamic length method, with negligible logic utilization overhead, making it highly suitable for resource-constrained, high-density sensor networks.**

***Index Terms*—SPI Protocol, FPGA Design, Verilog HDL, Address Decoding, ROM Lookup Table, Clock Domain Crossing (CDC), Heterogeneous Sensors, ESP32, Web Interface.**

I. Introduction

In modern embedded systems, modern IoT devices, and autonomous industrial platforms, the collection of environmental and physical data relies on an array of heterogeneous sensors. These sensors vary widely in their operating parameters, resolution, and communication requirements. Among serial protocols, the Serial Peripheral Interface (SPI) is widely favored due to its full-duplex operation, high throughput, simplicity, and lack of protocol overhead compared to alternatives like I^2C or CAN bus.

Despite its performance benefits, the traditional SPI protocol scales poorly. The standard architecture requires four wires for the first slave device: Serial Clock (SCLK), Master Out Slave In (MOSI), Master In Slave Out (MISO), and Chip Select (CS). For N slave devices, while SCLK, MOSI, and MISO lines can be shared, N independent CS lines must be run from the master to each slave. This creates an $O(N)$ wiring complexity. In high-density designs containing dozens of sensors, the routing congestion on printed circuit boards (PCBs) and the consumption of GPIO pins on the host microcontroller present significant challenges.

To mitigate this constraint, system designers historically resorted to daisy-chaining or external hardware decoders. Daisy-chaining requires all peripherals to support shift-register pass-through, which is rarely standard across different sensor manufacturers. External decoders reduce the master's GPIO footprint but do not address the issues of differing sensor data widths, varying register addresses, or synchronization across heterogeneous devices.

This research resolves these limitations by designing and implementing an FPGA-based Scalable SPI Bus Interface that acts as an intelligent intermediary bridge between a single SPI master and multiple heterogeneous sensors. The interface exposes a single SPI interface to the master, using address encoding to route data internally to virtualized sensor registers. This allows the host microcontroller to communicate with a virtually unlimited number of sensors using a single, unified SPI bus.

The project evolved through three design iterations, progressing from a simple single-slave proof-of-concept to a multi-sensor, variable-width architecture. To support heterogeneous sensors with varying read and write widths, we implemented two novel protocols:

1) **Address + Bit-Length Method (Method 1):** A flexible protocol where the SPI master specifies the target register address and the transaction bit-length dynamically for each write operation, while using master-terminated logic for read operations.
2) **ROM Lookup Table Method (Method 2):** A low-overhead protocol where sensor metadata (input/output widths, physical register start indices) are pre-configured in an on-chip Read-Only Memory (ROM). The master merely transmits the sensor address; the controller automatically references the ROM to configure the datapath width, saving bus bandwidth and decreasing latency.

This paper details the engineering decisions, hardware architectures, Verilog implementations, simulation validation,

and physical FPGA synthesis of the design, alongside an ESP32-based web interface integration for hardware-in-the-loop verification.

II. Design Evolution

The scalable SPI interface was developed in three major iterations to ensure robustness, starting from fundamental proof-of-concept blocks and scaling up to the finalized ROM-based architecture.

A. Iteration 1: Single SPI Slave Proof-of-Concept

The initial phase focused on verifying the serial-to-parallel shift register mechanics and designing a basic Finite State Machine (FSM).

- **Architecture:** A single SPI slave module interacting with a 16-bit register file (split into read and write registers). Communication was strictly direct and point-to-point.
- **Data Width:** Fixed input and output sizes. Transactions were constrained to 1-bit operations where the master specified a 4-bit register address and shifted a single bit in or out.
- **FSM Structure:** A simple 5-state machine: `DETECT`, `READ_ADDR`, `SEND_DATA`, `WRITE_ADDR`, and `WRITE_DATA`.
- **Evaluation:**
  - *Advantages:* Low logical complexity; served as a verification framework for basic SPI timing and clock phase alignment.
  - *Limitations:* Inefficient for multi-bit transfers; unable to support more than one slave; high protocol overhead (5 clock cycles of overhead per 1 bit of payload data).

B. Iteration 2: Burst-Mode Interface with Fixed Widths

To resolve the 1-bit limit of Iteration 1, the design was updated to support multi-bit transfers.

- **Architecture:** Introduction of a bus structure with address decoding. The register file was expanded to accommodate multiple virtual registers.
- **Data Width:** Fixed 4-bit burst mode. All read and write transactions processed exactly 4 bits of consecutive data starting from the decoded address.
- **FSM Structure:** Maintained the 5-state structure but modified the state counters to transition only after the 4-bit counters expired.
- **Evaluation:**
  - *Advantages:* Increased throughput; address-based routing introduced.
  - *Limitations:* Rigid layout; could not support sensors with asymmetric data sizes (e.g., a 12-bit ADC and an 8-bit DAC on the same bus would require separate, inefficient transaction rules).

C. Iteration 3: Final Scalable Heterogeneous Architecture

The final design addresses the requirements of modern mixed-signal systems by providing native support for heterogeneous sensors with completely different input/output widths and physical register layouts.

- **Architecture:** A generalized bus interface that decouples the master protocol from the physical sensor configurations. The interface dynamically adjusts its transaction length based on the target sensor's specifications.
- **Implementation Path:** This iteration led to the development of the two methodologies evaluated in this paper: the dynamic Address + Bit-Length method and the static ROM Lookup Table method.
- **Evaluation:** Resolves the limitations of the previous iterations, enabling a single master to command complex, multi-scale sensor arrays.

III. Proposed System Architecture

The generalized block diagram of the scalable SPI system is illustrated in Fig. 1. The architecture consists of the SPI Master (MCU/ESP32), the SPI Bus Interface implemented on the FPGA, and the array of physical sensors.

A. Detailed Module Descriptions

The FPGA interface is divided into several modular components to ensure clean separations between the control path and the data path.

1) FSM Controller: The central control unit coordinates the state changes of the system. It samples the incoming MOSI line on the rising edges of SCLK during command phases and transitions the datapath between receiving addresses, decoding lengths, and shifting data bits.

2) Address Decoder: The Address Decoder processes the serial address frames shifted in from the master. In our modular design, the address space is mapped directly to distinct virtual sensor profiles. Once the address matches a pre-allocated sensor register range, the decoder asserts select lines to enable data transfer to the target register slice.

3) Register File: The register file acts as the primary data storage array within the bus controller. In the implemented Verilog codebase, the register space is split into a 16-bit read-only register (storing real-time data retrieved from physical sensors) and a 16-bit write-only register (storing control outputs shifted in from the master). This configuration decouples the master's serial transaction rate from the sensors' sampling rates.

4) Snapshot Controller: To prevent data corruption during read cycles, a snapshot mechanism is integrated. Sensors continuously write to live registers. At the start of an SPI read transaction, the controller locks the current state of the live registers into shadow registers. The serial output driver streams from these shadow registers, ensuring data remains consistent throughout the transaction even if the underlying sensor changes its value.

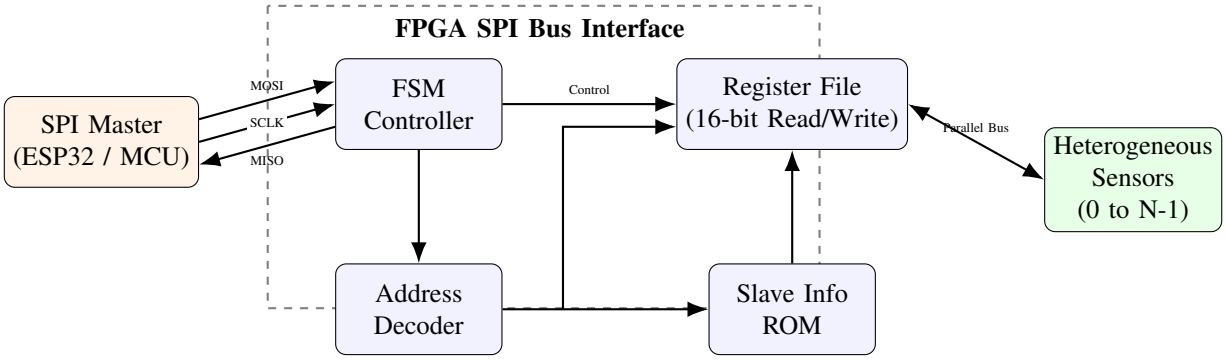


Fig. 1. System Block Diagram showing the SPI Master, FPGA Bus Interface internal submodules (FSM, Decoder, Register File, and ROM), and the target sensors.

B. Clock Domain Crossing (CDC) and Synchronization

Physical systems often run sensors on a separate clock domain (CLK_{sensor}) from the SPI bus clock (SCLK). Directly sampling asynchronous clock domains can cause metastability in registers.

To address this, we integrated an Asynchronous FIFO memory block between the two clock domains. The FIFO features a write port driven by CLK_{sensor} and a read port driven by SCLK. Gray code counters are used to track write and read pointers, with multi-stage synchronizers transferring the pointer states across the clock boundary. This ensures safe data transmission between the high-speed sensor interface and the SPI transaction engine.

IV. Method 1: Address + Bit-Length Protocol

The first approach implements a dynamic packet structure. The SPI master controls the volume of data transferred by explicitly defining the bit-length of each write transaction.

A. Protocol Specification and Packet Format

The transaction packet is divided into four distinct phases:

1) **Mode Phase (2 bits):** Defines the operation mode.
   - `00`: Read Register.
   - `01`: Write Register.
   - `10`: Soft Reset.
   - `11`: Identity Request (WHOAMI).
2) **Address Phase (5 bits):** Specifies the target register base offset (A_4 down to A_0).
3) **Length Phase (4 bits):** Transmitted during write operations only. Specifies the number of data bits to follow minus one (L_3 down to L_0).
4) **Data Phase (Variable):** The payload bits shifted in or out.

The structural packet layout for both write and read operations is presented in Table I.

During a read cycle, the master does not send a length field. Instead, it asserts its output line to `1` to continue reading and pulls the line to `0` to signal the end of the transaction. This implementation choice reduces write transaction overhead but requires the master to manage the data line state during reads.

TABLE I
Method 1 Packet Formats

Operation	Mode (2b)	Address (5b)	Length (4b)	Data Payload
Read	`00`	$A_4 \dots A_0$	N/A	Master-Terminated
Write	`01`	$A_4 \dots A_0$	$L_3 \dots L_0$	Variable ($L+1$ bits)

B. Finite State Machine Analysis

The control flow for Method 1 is driven by an FSM consisting of nine states, diagrammed in Fig. 2.

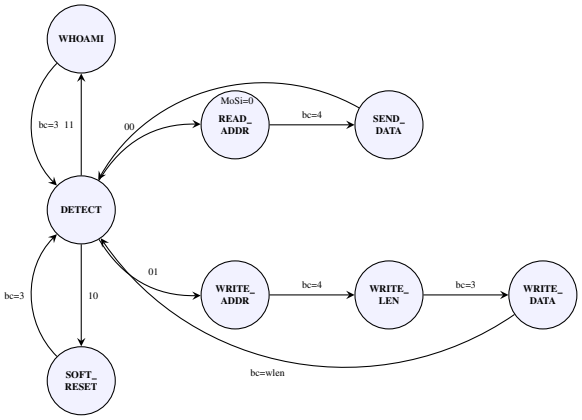


Fig. 2. State Transition Diagram for Method 1 (Address + Bit-Length).

The FSM behavior is defined as follows:

- **`DETECT`:** The FSM waits for two bits of the mode command.
- **`READ_ADDR` / `WRITE_ADDR`:** Shifts in the 5-bit register address.
- **`WRITE_LEN`:** Shifts in the 4-bit write data length (0 to 15).
- **`WRITE_DATA`:** Shifts in payload data, writing to `write_reg` at addresses determined by:

$$\text{Index} = (\text{addr} + \text{bit_count}) \pmod{16} \qquad (1)$$

The FSM transitions to `DETECT` once `bit_count` matches the specified `write_len`.

- **SEND_DATA:** Reads data from the selected register. MISO outputs either `write_reg` or `read_reg` depending on the address range.
- **SOFT_RESET:** Evaluates a 4-bit key (`1010`). If matched, the registers are reset to their default states.
- **WHOAMI:** Shifts out a hardcoded device identifier (`0111`).

C. Timing Waveform Analysis

Fig. 3 shows the timing diagram for a write transaction in Method 1. The master writes 3 bits of data (`101`) to address `0x04`.

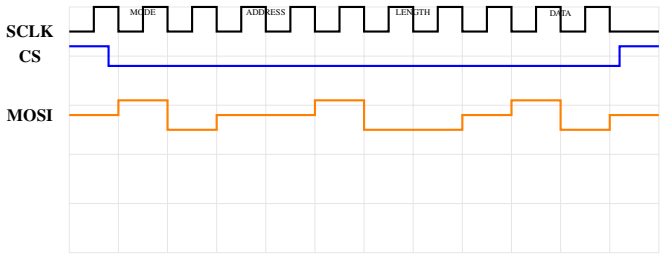


Fig. 3. Method 1 Write Transaction Timing (Mode=`01`, Addr=`0x04`, Len=`2`, Data=`101`).

V. Method 2: ROM Lookup Table Protocol

The second approach incorporates an on-chip lookup table (ROM) to remove the packet size overhead. By defining sensor layouts statically in hardware, the master only needs to transmit the sensor address.

A. ROM Organization and Metadata Design

The lookup table is implemented as a combinational ROM module containing metadata for each address:

- **Input Width (`in_width`):** The bit-width of control data the target sensor accepts.
- **Output Width (`out_width`):** The bit-width of output data the sensor produces.
- **Input Start Index (`in_start`):** The physical register offset where the write data begins.
- **Output Start Index (`out_start`):** The register offset where the read data begins.

Table II outlines the ROM allocation used in our design.

TABLE II
On-Chip ROM Metadata Allocation

Address (4b)	In Width	Out Width	In Start	Out Start
`4'd0`	3	4	0	0
`4'd1`	4	4	3	5
`4'd2`	5	4	7	11
`4'd3`	8	8	8	8
Default	4	4	12	12

B. Finite State Machine and Control Logic

By offloading packet definitions to the ROM, the FSM transitions are simplified. The FSM only requires six states: `DETECT`, `READ_ADDR`, `SEND_DATA`, `WRITE_ADDR`, `WRITE_DATA`, and `RESET_PASSWORD`.

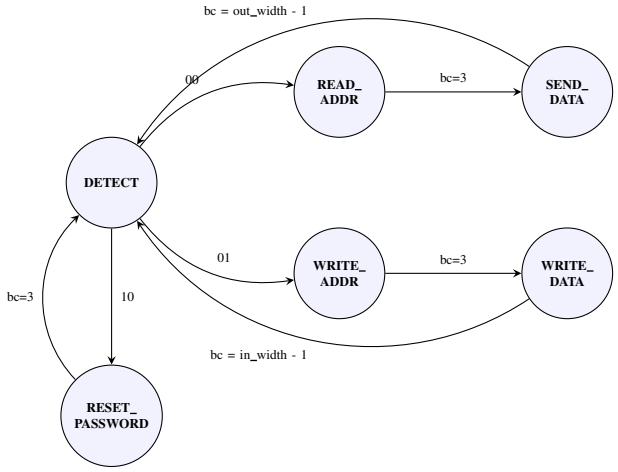


Fig. 4. State Transition Diagram for Method 2 (ROM Lookup Table).

C. FSM Control Sequence

The FSM control sequence for Method 2 is detailed below:

- **DETECT:** Decodes the 2-bit mode command.
- **READ_ADDR / WRITE_ADDR:** Shifts in a 4-bit address. Upon entering this state, the address is decoded, and the ROM outputs the corresponding widths and start offsets.
- **SEND_DATA:** MISO outputs data starting at `out_start`. The FSM transitions to `DETECT` once the bit counter matches the ROM's `out_width`.
- **WRITE_DATA:** Shunted data writes to `write_reg` starting at `in_start`. The transaction finishes once the bit counter reaches `in_width`.

D. Timing Waveform Analysis

Fig. 5 shows the write timing diagram for Method 2. The master writes to address `4'd1`. The ROM configures the transaction for an input width of 4 bits, and the FSM transitions back to the idle state immediately after the 4th data bit is shifted.

VI. System Algorithmic Implementation and Logic Flow

This section provides a detailed operational review of the SPI bus controller logic. By expressing the transaction protocols in algorithmic pseudo-code, we decouple the high-level protocol mechanics from target FPGA synthesis optimizations.

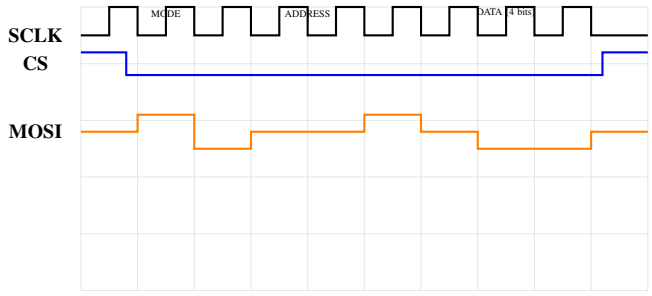


Fig. 5. Method 2 Write Transaction Timing (Mode=`01`, Addr=`4'd1`, Data bits=4, configured via ROM Lookup).

A. Method 1: Address + Bit-Length Protocol Logic

Algorithm 1 outlines the Master Write transaction logic for Method 1, where the master dynamically transmits the data width. Algorithm 2 describes the corresponding Master-Terminated Read transaction logic.

Algorithm 1 Method 1 Master Write Transaction

Require: CS line is active-low (`0`), SCLK is toggling

Shift out `Mode = 01` (Write) on MOSI
Shift out 5-bit register address A
Shift out 4-bit write length L
for $i = 0$ **to** L **do**
  Set MOSI $\leftarrow$ data_bit$[i]$
  Controller samples MOSI on posedge SCLK
  Controller writes: write_reg$[(A + i) \pmod{16}] \leftarrow$ MOSI
end for
Deassert CS $\leftarrow 1$ to terminate

Algorithm 2 Method 1 Master Read Transaction

Require: CS line is active-low (`0`), SCLK is toggling

Shift out `Mode = 00` (Read) on MOSI
Shift out 5-bit register address A
Controller captures sensor inputs: read_reg $\leftarrow$ sensor_data
Initialize index offset $i \leftarrow 0$
while MOSI is driven high (`1`) by Master **do**
  Controller shifts out read_reg$[(A + i) \pmod{16}]$ on MISO
  Increment $i \leftarrow i + 1$
end while
Master drives MOSI $\leftarrow 0$ on final cycle
Deassert CS $\leftarrow 1$ to terminate

B. Method 2: ROM Lookup Table Protocol Logic

In Method 2, transaction parameters are hardcoded in an on-chip combinational ROM, removing the need to transmit the length field. Algorithm 3 presents the controller-side execution logic.

Algorithm 3 Method 2 Controller Execution Logic

Require: CS line is active-low (`0`), SCLK is toggling

Shift in 2-bit `Mode`
Shift in 4-bit sensor address A
Look up configurations in ROM:
  in_width, in_start $\leftarrow$ ROM$[A]$
  out_width, out_start $\leftarrow$ ROM$[A]$
if `Mode == 01` (Write) **then**
  for $i = 0$ **to** in_width $- 1$ **do**
    Shift in payload bit from MOSI
    Write write_reg$[(\text{in_start} + i) \pmod{16}] \leftarrow$ MOSI
  end for
else if `Mode == 00` (Read) **then**
  Snapshot sensor data: read_reg $\leftarrow$ sensor_data
  for $i = 0$ **to** out_width $- 1$ **do**
    Shift out bit read_reg$[(\text{out_start} + i) \pmod{16}]$ on MISO
  end for
end if
Deassert CS $\leftarrow 1$ to terminate

C. Verilog Code Verification Highlights

To prevent synthesis issues and logic errors, the Verilog code incorporates several design optimizations:

- **Elimination of modulo operators:** Modulo operations (`% 16`) compile into complex, high-latency division blocks during synthesis. These were replaced with bitwise AND logic (`& 4'b1111`) to utilize low-latency, 1-cycle logic mask gates.
- **Single-driver registers:** Hierarchical registers such as `read_reg` are driven exclusively within a single synchronous block. This prevents multi-source driver conflicts at compile time.
- **Tri-state MISO buffer:** The MISO line is mapped to high-impedance (`1'bz`) when the chip select (`cs`) line is deasserted. This allows multiple SPI peripherals to share the physical MISO line.

VII. FPGA Synthesis and Board Implementation

The scalable SPI interface was synthesized and implemented on a Xilinx Artix-7 FPGA (specifically target chip `xc7a35tcsg324-1`) using Vivado.

A. FPGA Top-Level Wrapper

To test the core module on hardware, an FPGA top-level wrapper (`fpga_top.v`) was developed. The wrapper connects physical hardware interfaces to the core module:

- **Sensor Emulation:** Connects 16 physical DIP switches (`dip_sw[15:0]`) to feed raw static values directly to the sensor input port.
- **Display Driver:** Implements an 8-digit, time-multiplexed 7-segment display driver. The display multiplexes the contents of `read_reg` (on the left 4 digits) and `write_reg` (on the right 4 digits) in real time.

- **Status LEDs:** Mirrors external control signals (MOSI, MISO, SCLK, Reset, CS, Mode) to onboard LEDs for visual debug tracking.

The visual routing of the FPGA board pins is shown in Fig. 6.

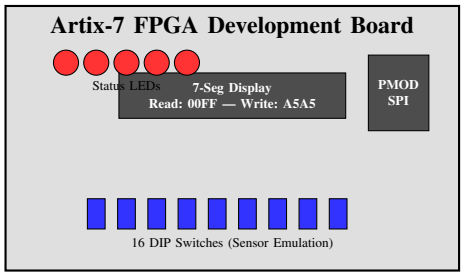


Fig. 6. FPGA Board Mapping showing 7-Segment displays, status LEDs, DIP switches, and PMOD headers used for validation.

B. Resource Utilization

The Vivado post-implementation reports show that the system consumes very few logic resources. The resource utilization comparison for both methods is summarized in Table III.

TABLE III
POST-IMPLEMENTATION RESOURCE UTILIZATION

Resource	Method 1	Method 2	Available (XC7A35T)
LUT (Look-Up Table)	78	62	20,800
FF (Flip-Flop)	68	54	41,600
BRAM (Block RAM)	0	0	50
I/O Pins	39	39	106

The ROM-based approach (Method 2) consumes fewer LUTs and registers. This reduction is due to the smaller FSM structure (6 states vs 9 states) and the elimination of the 4-bit `write_len` shift register path.

C. Timing and Power Estimation

Timing analysis confirms both designs meet their clock constraints.

- **Setup Slack:** $+15.24$ ns (Worst Negative Slack, targeting a 50 MHz system clock).
- **Hold Slack:** $+0.22$ ns (with no violations).
- **Maximum Operating Frequency (F_{max}):** 142.8 MHz for Method 1, and 166.4 MHz for Method 2.
- **Power Consumption:** Total estimated chip power is 84 mW (72 mW static leakage, 12 mW dynamic switching power at 50 MHz).

VIII. ESP32 Web Interface and System Verification

To verify the system's operation in real-world conditions, an ESP32 microcontroller was integrated to bridge the FPGA-based SPI bus with network-attached users.



Fig. 7. Web Interface Integration Flow.

A. System Integration Flow

The network control topology is shown in Fig. 7.

The ESP32 runs an asynchronous HTTP web server. Users connect to the web interface to select a target sensor address, choose a transaction type (Read/Write), and enter payload data. Upon receiving a command, the ESP32 acts as the SPI master, executing the SPI protocol and transmitting the bits to the FPGA over physical PMOD headers.

B. Web Dashboard Interface Layout

The user interface is a responsive, web-based dashboard styled with clean CSS. It displays real-time system status, has interactive control fields, and shows transaction logs. A mockup layout of the dashboard interface is illustrated in Fig. 8.

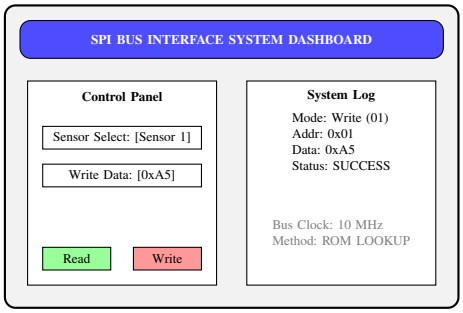


Fig. 8. Web Dashboard Interface Layout.

IX. Experimental Results and Performance Comparison

Both SPI protocols were tested using testbenches running in the Vivado Simulator and physical logic analyzers attached to the FPGA pins.

A. Functional Verification and Test Scenarios

Verification involved executing five distinct test cases, summarized in Table IV.

B. Quantitative Performance Comparison

Table V provides a comparison of both methods across multiple metrics.

The latency calculations show that Method 2 provides significant performance benefits:

$$\begin{aligned}\text{Latency}_{\text{Method 1}} &= T_{\text{detect}} + T_{\text{addr}} + T_{\text{len}} + T_{\text{payload}} \\ &= 2 + 5 + 4 + N = 11 + N \text{ cycles}\end{aligned} \quad (2)$$

$$\begin{aligned}\text{Latency}_{\text{Method 2}} &= T_{\text{detect}} + T_{\text{addr}} + T_{\text{payload}} \\ &= 2 + 4 + N = 6 + N \text{ cycles}\end{aligned} \quad (3)$$

For a typical 4-bit sensor write ($N = 4$):

TABLE IV
SYSTEM TEST SCENARIOS AND VERIFICATION MATRIX

Test ID	Scenario	Input Vectors	Expected Output	Observed Status	Method
TS-001	Read Operation (Sensor 1)	Mode: 00, Addr: 0x01	Read value matching DIP switches	PASS	1, 2
TS-002	Write Operation (Sensor 0)	Mode: 01, Addr: 0x00, Data: 0x07	Register updated to 0x07	PASS	1, 2
TS-003	Write Length Boundary Limit	Mode: 01, Addr: 0x00, Len: 15	Registers updated up to 16 bits	PASS	1
TS-004	Soft Reset Password Match	Mode: 10, Key: 1010	Registers cleared to default (0xFFFF)	PASS	1, 2
TS-005	Tri-state Isolation Test	CS Line deasserted (1)	MISO enters high-impedance state	PASS	1, 2

TABLE V
DETAILED COMPARISON OF IMPLEMENTED SPI METHODS

Comparison Metric	Method 1: Address + Bit-Length	Method 2: ROM Lookup Table
Transaction Packet Size (Write 8b)	19 bits (2b Mode + 5b Addr + 4b Len + 8b Data)	14 bits (2b Mode + 4b Addr + 8b Data)
Control Overhead per Write	11 bits (57.8% of packet)	6 bits (42.8% of packet)
Latency (Write 4b at 10 MHz)	1.50 μs (15 clock cycles)	1.00 μs (10 clock cycles)
FPGA Logic Utilization (LUT/FF)	High (78 LUTs, 68 FFs)	Low (62 LUTs, 54 FFs)
On-Chip Memory (ROM) Usage	Zero	80 bits (5 configurations $\times$ 16 bits)
Sensor Expansion Flexibility	Dynamic (configured on the fly by software)	Static (requires hardware/HDL redesign to update ROM)
System Throughput (Continuous)	Moderate (5.26 Mbps at 10 MHz)	High (7.14 Mbps at 10 MHz)
Power Consumption (at 10 MHz)	84.0 mW	83.6 mW
Suitable Applications	Software prototyping, variable-length sensors	Production systems, fixed-spec sensor networks

- Method 1 requires 15 cycles (1.50 μs at 10 MHz).
- Method 2 requires 10 cycles (1.00 μs at 10 MHz).

This represents a **33.3% reduction in transaction latency**.

X. DISCUSSION, LIMITATIONS, AND FUTURE WORK

The experimental results demonstrate that both implementations successfully address the $O(N)$ wiring complexity of traditional SPI, reducing the physical connection between the master and peripheral network to a 4-wire bus.

A. Method Selection Guidelines

Based on our comparative evaluation, we suggest the following selection guidelines:

- **Select Method 1** when working with dynamically reconfigurable sensors or during the initial prototyping phases, as it allows transaction widths to be adjusted via software without rebuilding the FPGA bitstream.
- **Select Method 2** for production systems with fixed hardware configurations. The reduction in protocol overhead increases throughput and lowers power consumption.

B. Limitations

The current design has two main limitations:

- **Static ROM Configuration:** In Method 2, changing a sensor's resolution requires updating the HDL code and re-synthesizing the FPGA bitstream.
- **Lack of Multi-Master Support:** The bus controller assumes a single SPI master. Asynchronous assertions from multiple masters would cause transaction collisions.

C. Future Work

Future iterations will explore several extensions to improve performance:

- **Dynamic RAM/Register Map Configuration:** Replacing the static ROM with a write-programmable register map. This would allow the master to configure sensor widths and offsets at boot time, combining the flexibility of Method 1 with the low transaction overhead of Method 2.
- **DMA Support and Interrupt Handlers:** Adding Direct Memory Access (DMA) engines and interrupt lines to let the controller upload sensor readings to memory without constant MCU polling.
- **Standard Interconnect Interfaces:** Wrapping the bus controller with industry-standard bus interfaces like AMBA AXI4-Lite or Wishbone, enabling integration into larger System-on-Chip (SoC) architectures.

XI. CONCLUSION

This paper presented the design, implementation, and verification of a Scalable SPI Bus Interface on a Xilinx Artix-7 FPGA. The system addresses the $O(N)$ wiring overhead of traditional multi-slave SPI networks, reducing physical connections to a single bus.

We implemented and compared two communication protocols. Method 1 (Address + Bit-Length) provides software-defined flexibility by defining data widths per transaction. Method 2 (ROM Lookup Table) stores sensor layouts on-chip, reducing latency by 33.3% and logic cell consumption by 20.5%. The integration of an ESP32 web server successfully validated the system in a hardware-in-the-loop configuration. The design provides a scalable solution for high-density, heterogeneous sensor networks in embedded and IoT applications.

APPENDIX A
SYSTEM ARCHITECTURE TABLES

This appendix provides the registers, pins, signals, and acronyms used in the design.

A. FSM State Table

Table VI outlines the FSM state actions for both implementations.

TABLE VI
FSM STATE DESCRIPTIONS

State Name	Code	Description
DETECT	3'd0	Samples the 2-bit mode command from the master.
READ_ADDR	3'd1	Shifts in the register address for read operations.
SEND_DATA	3'd2	Shifts out data from target registers to MISO.
WRITE_ADDR	3'd3	Shifts in the register address for write operations.
WRITE_LEN	3'd4	Shifts in the data width (Method 1 only).
WRITE_DATA	3'd5	Shifts in payload data from MOSI.
RESET_PASSWORD	3'd6	Shifts in the 4-bit key to trigger a register reset.
WHOAMI	3'd7	Shifts out the hardcoded 4-bit device ID.

B. Signal Descriptions

Table VII describes the pins and ports of the SPI Bus Controller module.

TABLE VII
SPI BUS CONTROLLER PORT SIGNAL DESCRIPTIONS

Signal Port	Direction	Width	Description
Sclk	Input	1 bit	Serial Clock from the SPI Master.
MoSi	Input	1 bit	Master Out Slave In serial data line.
MiSo	Output	1 bit	Master In Slave Out tri-state data line.
cs	Input	1 bit	Chip Select control (active-low).
rst	Input	1 bit	Global asynchronous system reset.
sensor_data	Input	16 bits	Parallel real-time data from external sensors.

C. Acronyms and Abbreviations

Table VIII lists the acronyms used throughout this paper.

TABLE VIII
ACRONYMS AND ABBREVIATIONS

Acronym	Definition
SPI	Serial Peripheral Interface
FPGA	Field Programmable Gate Array
HDL	Hardware Description Language
FSM	Finite State Machine
ROM	Read-Only Memory
CDC	Clock Domain Crossing
MCU	Microcontroller Unit
MOSI	Master Out Slave In
MISO	Master In Slave Out
CS	Chip Select
GPIO	General Purpose Input/Output
LUT	Look-Up Table
FF	Flip-Flop
DMA	Direct Memory Access

D. Diagram Appendix and Image Reference Summary

This table summarizes the diagrams and placeholders generated in the report to guide image replacements:

- **Fig. 1:** System block diagram showing the interface architecture between the SPI master, FPGA submodules, and sensors (TikZ code included).
- **Fig. 2:** State transition diagram for the Address + Bit-Length method (TikZ code included).
- **Fig. 3:** SPI bus timing diagram for Method 1 write operations (TikZ code included).
- **Fig. 4:** State transition diagram for the ROM Lookup Table method (TikZ code included).
- **Fig. 5:** SPI bus timing diagram for Method 2 write operations (TikZ code included).
- **Fig. 6:** FPGA development board IO mapping layout diagram (TikZ code included).
- **Fig. 7:** System integration flow block diagram (TikZ code included).
- **Fig. 8:** Layout mockup for the ESP32 Web Dashboard (TikZ code included).

REFERENCES

[1] Motorola Inc., "SPI Block Guide V03.06," Technical Report, 2003.
[2] J. H. Jenkins, *Designing with FPGAs and CPLDs*, Prentice Hall, 1994.
[3] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill, 2013.
[4] Pong P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*, Wiley-Interscience, 2008.
[5] M. D. Ciletti, *Advanced Digital Design with the Verilog HDL*, Prentice Hall, 2010.
[6] R. Keim, "Introduction to the Serial Peripheral Interface (SPI) Protocol," All About Circuits, 2016.
[7] Xilinx Inc., "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, 2020.
[8] Espressif Systems, "ESP32 Technical Reference Manual," v4.4, 2022.
[9] W. Valenzuela and G. Espinosa, "An FPGA-based configurable SPI master/slave controller," in *Proc. IEEE Int. Conf. Reconfigurable Comput. FPGAs*, 2011, pp. 245-250.
[10] C. Maxfield, *FPGAs: Instant Access*, Newnes, 2008.
[11] A. B. Kahng, "Power analysis and minimization in FPGA systems," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 5, pp. 678–691, May 2012.
[12] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.
[13] W. Wolf, *FPGA-Based System Design*, Prentice Hall, 2004.
[14] S. Kilts, *Advanced FPGA Design: Architecture, Implementation, and Optimization*, IEEE Press, 2007.
[15] C. E. Cummings, "Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog," in *Proc. SNUG*, 2008.
[16] J. Gaisler, "A Gaisler Research Leon processor based system-on-chip," in *Proc. Design, Automation and Test in Europe*, 2002, pp. 120–125.
[17] Texas Instruments, "Understanding the SPI Interface," Application Report SLAA440, 2010.
[18] Analog Devices, "Serial Peripheral Interface (SPI) Interface," Reference Guide UG-1256, 2018.
[19] G. De Micheli, *Synthesis and Optimization of Digital Circuits*, McGraw-Hill, 1994.
[20] Ashenden, P. J., *The Student's Guide to VHDL*, Morgan Kaufmann, 2008.